



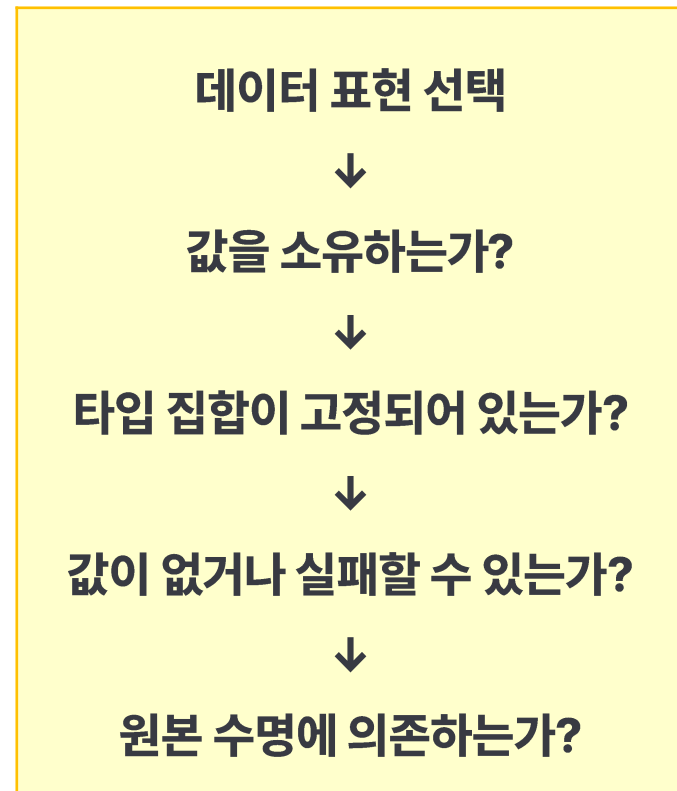
14. 유틸리티 타입과 데이터 표현

Heesung Oh

<https://github.com/3dapi>



- 여러 값을 묶는 타입
- 값의 부재 표현
- 여러 타입 중 하나를 저장하는 타입
- 실행 시간 타입 저장
- 비 소유 참조와 뷰
- 비트와 바이트 표현
- 타입 안전한 상태 표현





14.1 std::pair와 std::tuple

- std::pair는 두 값을 하나로 묶음
- std::tuple은 여러 값을 하나로 묶음
- 짧은 중간 결과나 범용 코드에 적합
- 원소 의미가 중요하면 이름 있는 구조체가 더 명확

```
#include <string>
#include <utility>
#include <tuple>
```

```
std::pair<int, std::string> item{1001, "Potion"};
std::tuple<int, std::string, int> player{1, "Knight", 100};
std::cout << item.first << '\n';
std::cout << item.second << '\n';
```

타입	용도
<code>std::pair<T1, T2></code>	값 두 개
<code>std::tuple<Ts...></code>	값 여러 개
구조체	의미가 오래 유지되는 데이터





- std::get<I>()로 위치 기반 접근
- 같은 타입이 한 번만 등장하면 타입 기반 접근 가능
- 같은 타입이 여러 번 있으면 타입 기반 접근 불가

```
std::tuple<int, std::string, int> player{1, "Knight", 100};  
int id = std::get<0>(player);  
std::string name = std::get<1>(player);  
int hp = std::get<2>(player);  
  
std::tuple<int, std::string, double> data{10, "Potion", 2.5};  
std::string& text = std::get<std::string>(data);  
std::pair<int, std::string> item{1001, "Potion"};  
  
int itemId = std::get<0>(item);  
std::string& itemName = std::get<1>(item);
```





14.1.3 std::tie와 구조적 바인딩

```
int id = 0;  
std::string name;  
int hp = 0;  
  
std::tie(id, name, hp) = std::make_tuple(1, std::string{"Knight"}, 100);  
std::tie(id, std::ignore, hp) = std::make_tuple(2, std::string{"Mage"}, 80);  
  
auto [playerId, playerName, playerHp] = std::make_tuple(1,  
std::string{"Knight"}, 100);
```

선언	의미
auto [a, b]	원소를 새 변수로 복사 또는 이동
auto& [a, b]	원본 원소 참조
const auto& [a, b]	원본 원소 읽기 전용 참조

- 구조적 바인딩은 튜플, pair, map 순회에서 자주 사용





14.1.4 여러 반환값과 std::apply

```
std::pair<int, int> DivideWithRemainder(int value, int divisor)
{
    return {value / divisor, value % divisor};
}
```

```
auto [quotient, remainder] = DivideWithRemainder(17, 5);
```

- 두 값의 의미가 문맥에서 분명하면 pair 사용 가능
- 의미가 중요하거나 여러 곳에서 사용하면 구조체가 더 명확

```
struct DivisionResult
{
    int quotient;
    int remainder;
};

int Add(int left, int right)
{
    return left + right;
}
```

```
std::tuple<int, int> arguments{10, 20};
int result = std::apply(Add, arguments);
```

- 튜플 원소를 함수 인수로 펼쳐 호출



- 값이 있거나 없는 상태를 타입으로 표현
- 검색 실패, 설정 누락, 입력 부재에 적합
- 특별한 센티널 값을 사용하지 않아도 됨
- 실패 이유까지 표현하지는 않음

```
#include <optional>
#include <vector>

std::optional<std::size_t> FindIndex(const std::vector<int>& values, int target)
{
    for (std::size_t index = 0; index < values.size(); ++index)
    {
        if (values[index] == target)
        {
            return index;
        }
    }
    return std::nullopt;
}
```

- std::nullopt는 값이 없는 상태





14.2.1 optional 사용

```
auto index = FindIndex(values, 30);  
if (index)  
{  
    std::cout << "index: " << *index << '\n';  
}  
else  
{  
    std::cout << "not found\n";  
}
```

표현	의미
optional<T>	T 값 또는 빈 상태
std::nullopt	빈 상태
if (value)	값 존재 여부 검사
*value	저장된 값 접근
value()	값 접근, 없으면 예외
value_or(default)	값 또는 기본값 반환

- **operator***, **operator->**는 값 존재를 전제로 사용
- 값이 없을 수 있음을 호출부에 강제



14.2.2 reset과 emplace

```
std::optional<int> score;  
std::optional<int> port = std::nullopt;
```

```
score = 100;  
score = 200;  
score = std::nullopt;
```

```
std::optional<std::string> text;
```

```
text.emplace(4, 'A');  
text.emplace("updated");  
text.reset();
```

동작	의미
기본 생성	값 없음
값 대입	값 저장
std::nullopt 대입	빈 상태
reset()	저장된 값 파괴
emplace()	내부에 새 값 직접 생성

- **emplace()**는 기존 값이 있으면 파괴 후 새 값 생성
- **optional**은 내부에 값을 직접 소유



14.2.3 값 접근과 기본값

```
std::optional<std::string> name = "Alice";  
if (name.has_value())  
{  
    std::cout << *name << '\n';  
    std::cout << name->size() << '\n';  
}  
  
try  
{  
    std::cout << name.value() << '\n';  
}  
catch (const std::bad_optional_access&)  
{  
    std::cout << "no value\n";  
}  
  
std::optional<int> port;  
int selectedPort = port.value_or(8080);
```

- `value_or()`는 참조가 아니라 값을 반환
- 큰 객체 기본값은 복사나 이동 비용 가능
- 값 유무에 따라 작업이 다르면 조건문이 더 명확



14.2.4 포인터와 optional의 차이

표현	값의 위치	소유 관계	빈 상태
T*	외부 객체	보통 비소유	nullptr
std::unique_ptr<T>	동적 객체	단독 소유	nullptr
std::shared_ptr<T>	동적 객체	공유 소유	nullptr
std::optional<T>	optional 내부	값 자체를 소유	std::nullopt

```
std::optional<Player> player;  
player.emplace("Knight", 100);  
Player* FindPlayer(std::vector<Player>& players, int id);
```

- 기존 객체를 가리키는 검색 결과는 포인터가 자연스러움
- 독립된 값의 부재는 optional<T>가 자연스러움
- C++20 std::optional<T>는 참조 타입을 원소로 사용할 수 없음



- 정해진 여러 타입 중 하나를 저장
- 가능한 타입 집합이 컴파일 시점에 고정
- 현재 활성 대안만 값을 가짐
- 타입별 처리를 std::visit()으로 연결

```
#include <variant>
```

```
#include <string>
```

```
std::variant<int, double, std::string> value;
```

```
value = 10;
```

```
value = 3.5;
```

```
value = std::string{"hello"};
```

```
variant<int, double, string>
```



한 시점에는 하나의 대안만 활성

- 기본 생성 시 첫 번째 대안 타입의 기본값 저장
- 첫 대안이 기본 생성 불가능하면 기본 생성 불가



- std::monostate는 데이터가 없는 대안 타입
- variant의 명시적 빈 상태로 사용 가능
- 첫 대안이 기본 생성 불가할 때도 활용

```
using Result = std::variant<std::monostate, int, std::string>;  
Result result;  
if (std::holds_alternative<std::monostate>(result))  
{  
    std::cout << "empty\n";  
}
```

표현	의미
std::monostate	사용자가 정의한 정상 빈 대안
std::nullopt	optional의 값 없음
valueless_by_exception()	대안 교체 실패로 생길 수 있는 예외 상태

- 단순히 값이 있거나 없으면 optional<T>가 더 간단



14.3.3 variant 값 확인과 접근

```
std::variant<int, double, std::string> value = 3.5;
std::cout << value.index() << '\n';
if (std::holds_alternative<double>(value))
{
    std::cout << "double\n";
}
double number = std::get<double>(value);
double sameNumber = std::get<1>(value);
if (double* number = std::get_if<double>(&value))
{
    std::cout << *number << '\n';
}
```

함수	역할
index()	활성 대안 위치
holds_alternative<T>()	타입 활성 여부
get<T>(), get<I>()	참조 접근, 실패 시 예외
get_if<T>()	포인터 접근, 실패 시 nullptr



```
std::variant<int, double, std::string> value = std::string{"hello"};  
std::visit([](const auto& stored)  
{  
    std::cout << stored << '\n';  
},  
value);
```

- 활성 대안 타입에 맞게 호출
- 모든 대안에서 본문이 컴파일 가능해야 함

```
std::visit([](const auto& stored) {  
    using T = std::remove_cvref_t<decltype(stored)>;  
  
    if constexpr (std::is_same_v<T, int>) {  
        std::cout << "integer\n";  
    }  
    else if constexpr (std::is_same_v<T, double>) {  
        std::cout << "real\n";  
    }  
    else {  
        std::cout << "text\n";  
    }  
}, value);
```





- 상태마다 필요한 데이터가 다르면 variant가 유용
- 서로 모순되는 상태 조합을 줄임
- 한 시점에는 하나의 상태만 활성화

```
struct Idle {};  
struct Moving {  
    float speed;  
};  
struct Attacking {  
    int targetId;  
    int damage;  
};
```

```
using CharacterState = std::variant<Idle, Moving, Attacking>;
```

```
CharacterState state = Idle{};  
state = Moving{3.5f};  
state = Attacking{42, 15};
```

Idle	→ 추가 데이터 없음
Moving	→ speed
Attacking	→ targetId, damage

14.3.6 valueless_by_exception

- variant는 보통 하나의 대안을 저장
- 대안 교체 중 예외가 발생하면 어떤 대안도 저장하지 못할 수 있음
- 이 상태는 `valueless_by_exception()`으로 확인

```
if (value.valueless_by_exception())  
{  
    std::cout << "no active alternative\n";  
}  
  
if (value.index() == std::variant_npos)  
{  
    std::cout << "invalid variant state\n";  
}
```

- 일반 코드에서 자주 만나는 상태는 아님
- `variant`가 절대로 비어 있지 않다고 단정하지 않음
- `std::monostate`와 의미가 다름

monostate



정상 대안

valueless_by_exception



대안 교체 실패로 인한 예외 상태



14.3.7 상속 기반 다형성과 variant

기준	상속 기반 다형성	std::variant
타입 집합	새 파생 타입으로 확장	대안 목록에 고정
호출 방식	가상 함수	std::visit()
저장 방식	보통 포인터·참조	값 직접 저장 가능
공통 인터페이스	기반 클래스에 정의	방문자가 연산 정의
새 연산 추가	기반 인터페이스 변경 가능	방문자 추가
새 타입 추가	파생 클래스 추가	variant와 방문자 수정

```
class Shape
{
public:
    virtual ~Shape() = default;
    virtual double Area() const = 0;
};
using ShapeValue = std::variant<Circle, Rectangle, Triangle>;
```

- 타입이 계속 추가되면 상속이 자연스러움
- 타입 집합이 고정되어 있으면 variant가 단순할 수 있음



- 실행 시간에 저장 타입을 결정
- 하나의 객체에 여러 타입을 차례로 저장 가능
- 저장 타입을 꺼낼 때 타입 확인 필요
- C++20에서는 복사 가능한 타입 저장

```
#include <any>
#include <string>

std::any value = 10;
value = 3.5;
value = std::string{"hello"};
// std::any pointer = std::make_unique<int>(10); // C++20 저장 불가
```

타입	저장 타입 집합
<code>std::variant<Ts...></code>	컴파일 시점에 고정
<code>std::any</code>	실행 시간에 결정

- 설정 저장소, 편집기 속성, 확장 모듈 경계에서 사용 가능



14.4.2 빈 상태와 저장 타입 확인

```
std::any value;  
if (!value.has_value())  
{  
    std::cout << "empty\n";  
}  
  
value = std::string{"hello"};  
value.reset();  
  
value.emplace<std::string>(5, '*');  
if (value.type() == typeid(std::string))  
{  
    std::cout << "string stored\n";  
}
```

- 빈 any의 type()은 typeid(void)
- type().name() 문자열에 프로그램 로직 의존 금지

함수	역할
has_value()	저장 객체 존재 여부
reset()	저장 객체 파괴 후 빈 상태
emplace<T>()	지정 타입 객체 생성
type()	저장 타입의 type_info 반환



- 값 또는 참조 형식 실패 → 예외
- 포인터 형식 실패 → nullptr
- 타입이 불확실하면 포인터 형식이 분기 코드와 잘 맞음

```
std::any value = std::string{"hello"};
std::string text = std::any_cast<std::string>(value);
std::string& text = std::any_cast<std::string&>(value);

text += " world";
try
{
    int number = std::any_cast<int>(value);
}
catch (const std::bad_any_cast&)
{
    std::cout << "type mismatch\n";
}
if (std::string* text = std::any_cast<std::string>(&value))
{
    std::cout << *text << '\n';
}
```





14.4.4 variant와 any의 차이

- 가능한 타입 집합을 알고 있으면 variant 우선 검토
- 외부 모듈이 임의 타입을 등록하면 any가 유연
- any를 과도하게 쓰면 타입 안전성이 약해질 수 있음

기준	std::variant	std::any
저장 타입	템플릿 인수로 제한	실행 시간 결정
처리 누락	방문자 컴파일 오류로 발견 가능	실행 시간 검사 필요
타입 정보	대안 목록이 타입에 드러남	std::any만 보아서는 모름
이동 전용 타입	대안 조건이 허용하면 가능	C++20에서는 저장 불가
용도	닫힌 타입 집합	개방된 속성 저장소

```
using SettingValue = std::variant<int, double, std::string>;  
std::any dynamicValue;
```



- 참조를 복사 가능하고 대입 가능한 객체 형태로 감쌈
- 컨테이너에 참조 의미를 저장할 때 사용
- 대상을 소유하지 않음
- 대상 객체의 수명에 의존

```
#include <functional>
```

```
int value = 10;  
std::reference_wrapper<int> reference = std::ref(value);  
reference.get() = 20;  
std::cout << value << '\n';  
int& alias = reference;  
alias = 30;  
auto copied = reference;  
copied.get() = 40;
```

- 래퍼를 복사해도 원본 값은 복사하지 않음
- 같은 객체를 가리키는 래퍼가 만들어짐





14.5.2 컨테이너와 참조

```
struct Player
```

```
{  
    string name;  
    int hp;  
};
```

```
Player knight{"Knight", 100};  
Player mage{"Mage", 80};  
Player archer{"Archer", 90};
```

```
vector<std::reference_wrapper<Player>> party
```

```
{  
    knight,  
    mage,  
    archer  
};
```

```
for (Player& player : party)  
{  
    player.hp += 10;  
}
```

표현	특징
<code>std::vector<Player></code>	Player 객체를 소유
<code>std::vector<Player*></code>	포인터 값 저장, nullptr 가능
<code>std::vector<reference_wrapper<Player>></code>	기존 객체 참조, 빈 참조 없음

- 빈 참조가 필요하면 포인터 또는 `optional<reference_wrapper<T>>` 검토



14.5.3 std::ref, std::cref와 수명

```
int score = 100;  
auto scoreReference = std::ref(score);  
const std::string name = "Alice";  
auto nameReference = std::cref(name);  
// nameReference.get() += "!"; // 오류
```

```
// 값 전달 템플릿에 참조 의미 전달  
template <typename Function>  
void ExecuteTwice(Function function)  
{  
    function();  
    function();  
}
```

```
Counter counter;  
ExecuteTwice(std::ref(counter));
```

```
// 위험한 반환  
std::reference_wrapper<int> MakeInvalidReference()  
{  
    int local = 10;  
    return std::ref(local);  
}
```

- reference_wrapper는 대상을 소유하지 않음
- 대상 소멸, 컨테이너 재할당 후 사용 금지



- 연속 메모리 구간의 시작 위치와 원소 수를 묶는 비소유 범위
- 원소를 소유하거나 복사하지 않음
- C 배열, std::array, std::vector를 같은 인터페이스로 받을 수 있음
- C++20

```
#include <span>

void PrintValues(std::span<const int> values)
{
    for (int value : values)
    {
        std::cout << value << ' ';
    }
}
```

```
int first[]{10, 20, 30};
std::array<int, 3> second{40, 50, 60};
std::vector<int> third{70, 80, 90};

PrintValues(first);
PrintValues(second);
PrintValues(third);
```

- std::list는 연속 메모리가 아니므로 span 불가



14.6.1 span 기본 사용

- `size()` → 원소 수
- `empty()` → 빈 범위 여부
- `data()` → 시작 주소
- `operator[]`, `front()`, `back()` → 원소 접근
- C++20 `std::span::operator[]`는 범위 검사 없음

```
int values[]{10, 20, 30};  
std::span<int> view{values};  
std::size_t count = view.size();  
bool isEmpty = view.empty();  
int* firstAddress = view.data();
```

```
view[0] = 100;  
view.front() = 200;  
view.back() = 300;
```

```
void DoubleValues(std::span<int> values)  
{  
    for (int& value : values)  
    {  
        value *= 2;  
    }  
}
```

- 수정 가능한 `span`은 원본 원소 변경



14.6.3 정적 크기와 동적 크기

- 두 번째 템플릿 인수는 원소 수 extent
- 생략하면 `std::dynamic_extent`
- 정적 크기는 크기 요구를 타입에 포함

```
std::span<int> dynamicView{values};  
void NormalizeColor(std::span<float, 4> color)  
{  
    // RGBA 네 원소  
}  
  
std::array<float, 4> color{1.0f, 0.5f, 0.25f, 1.0f};  
NormalizeColor(color);  
std::array<float, 3> position{1.0f, 2.0f, 3.0f};  
  
// NormalizeColor(position); // 오류
```

span	크기
<code>std::span<T></code>	실행 시간 크기
<code>std::span<T, N></code>	컴파일 시점 고정 크기



14.6.4 span의 const와 원소의 const

- span 객체는 상수
- 원소 타입은 int
- 원소 수정 가능

```
int values[]{10, 20, 30};  
const std::span<int> fixedView{values};  
fixedView[0] = 100;
```

```
std::span<const int> readOnlyView{values};  
// readOnlyView[0] = 100; // 오류
```

선언	span 재 대입	원소 수정
std::span<int>	가능	가능
const std::span<int>	불가	가능
std::span<const int>	가능	불가
const std::span<const int>	불가	불가

- 읽기 전용 인터페이스는 **std::span<const T>** 사용



```
std::array<int, 6> values{10, 20, 30, 40, 50, 60};  
std::span<int> all{values};  
auto firstThree = all.first<3>();  
auto lastTwo = all.last<2>();  
auto middle = all.subspan(1, 3);
```

firstThree → 10 20 30

lastTwo → 50 60

middle → 20 30 40

middle[0] = 200; // values[1] 변경

// 잘못된 반환

```
std::span<int> MakeInvalidSpan()  
{  
    int values[]{10, 20, 30};  
    return values;  
}
```

- span은 원본 메모리를 소유하지 않음
- 원본 배열은 함수 종료 시 소멸





14.6.6 span 무효화 (Invalid)

```
std::vector<int> values;  
values.reserve(3);
```

```
values.push_back(10);  
values.push_back(20);  
values.push_back(30);
```

```
std::span<int> view{values};  
values.push_back(40); // 재할당 가능
```

- 재할당 발생 시 기존 원소 주소 변경
- span의 data()는 더 이상 유효하지 않음

```
std::vector<int> values{10, 20, 30};  
values.reserve(10);
```

```
std::span<int> view{values};  
values.push_back(40);  
std::cout << view.size() << '\n'; // 3
```

- span은 생성 시점의 범위 크기를 유지
- 새 원소까지 포함하려면 span을 다시 생성
- span을 멤버에 저장하거나 반환할 때는 원본 소유 객체의 수명 조건이 필요



14.7 std::string_view

- 연속된 문자 구간을 소유하지 않고 참조
- 문자열 리터럴, std::string, 문자 배열을 복사 없이 받을 수 있음
- 포인터와 길이를 저장
- 부분 문자열을 저렴하게 표현
- C++17

```
#include <string_view>

void PrintName(std::string_view name)
{
    std::cout << name << '\n';
}

PrintName("Alice");
std::string name = "Bob";
PrintName(name);

std::string_view text = "Hello, World";
std::cout << text.size() << '\n';
std::cout << text[0] << '\n';
```

- 읽기 전용 문자열 매개변수에 적합



14.7.2 문자열 복사 방지

```
bool StartsWithA(std::string text)
{
    return !text.empty() && text.front() == 'A';
}
```

- 값 매개변수는 문자열 복사 가능

```
bool StartsWithA(std::string_view text)
{
    return !text.empty() && text.front() == 'A';
}
```

- 읽기만 하면 string_view로 복사 방지 가능

```
class Player
{
public:
    explicit Player(std::string_view name)
        : name(name)
    {
    }
private:
    std::string name
}
```

- 매개변수는 비소유 뷰
- 멤버 std::string은 문자를 복사해 소유
- 보관해야 할 문자열은 소유 타입으로 저장



```
std::string_view path = "assets/characters/player.png";  
std::string_view folder = path.substr(0, 17);  
std::string_view fileName = path.substr(18);
```

- **substr()**는 원본 문자를 복사하지 않고 부분 뷰 반환

```
std::pair<std::string_view, std::string_view>  
SplitOnce(std::string_view text, char delimiter)  
{  
    std::size_t position = text.find(delimiter);  
    if (position == std::string_view::npos)  
    {  
        return {text, {}};  
    }  
    return { text.substr(0, position), text.substr(position + 1) };  
}
```

```
auto [key, value] = SplitOnce("width=1920", '=');
```

- 반환된 두 뷰는 호출 인수의 문자 저장 공간을 참조



- `string_view`가 가리키는 구간은 널 문자로 끝난다고 보장할 수 없음
- `data()`를 C 문자열 함수에 바로 전달하면 뷰의 경계를 넘을 수 있음

```
std::string text = "hello world";  
std::string_view word{text.data(), 5};
```

```
// std::printf("%s\n", word.data()); // hello world가 출력될 수 있음  
std::cout << word << '\n'; // hello
```

```
// C API에 전달
```

```
std::string nullTerminated{word};  
UseCString(nullTerminated.c_str());
```

- `operator<<`는 길이를 사용해 올바른 범위만 출력
- C API가 널 종료 문자열을 요구하면 소유 문자열로 변환



```
std::string_view name = std::string{"Alice"};
```

- 문장이 끝나면 임시 std::string 소멸
- name은 소멸한 문자열을 가리킴

```
std::string_view MakeInvalidName()  
{  
    std::string name = "Alice";  
    return name;  
}
```

지역 문자열은 함수 종료 시 소멸

```
std::string path = "assets/player.png";  
std::string_view fileName = FileName(path);
```

fileName은 path보다 오래 사용할 수 없음

```
std::string text = "short";  
std::string_view view = text;  
text += "text that may cause reallocation";
```

- 원본 string 재할당 시 기존 뷰 무효화 가능
- 뷰는 문자열 스냅샷이 아님



- 정수 비트 연산은 플래그, 마스크, 프로토콜 필드에 사용
- 원시 바이트를 다룰 때는 숫자 값과 객체 표현을 구분
- 표준 라이브러리는 바이트와 비트 표현 도구 제공

도구	역할
<code>std::byte</code>	메모리 한 바이트 표현
<code>std::bitset<N></code>	고정 크기 비트 집합
<code><bit></code> 함수	비트 개수, 거듭제곱, 순환 이동 등
<code>std::bit_cast</code>	객체 표현을 같은 크기 타입으로 복사
<code>std::endian</code>	기본 바이트 순서 확인

수치값 ≠ 메모리에 놓인 바이트 표현



- 메모리 한 바이트를 의미
- 일반 정수처럼 덧셈이나 곱셈을 수행하지 않음
- 비트 연산은 가능

```
#include <cstdint>  
std::byte value{0x2A};
```

```
std::byte flags{0b0000'0101};  
flags |= std::byte{0b0000'1000};  
flags &= std::byte{0b0000'1111};
```

```
unsigned int number = std::to_integer<unsigned int>(flags);  
std::vector<std::byte> buffer(1024);
```

- 바이너리 파일
- 네트워크 패킷
- 직렬화 버퍼
- 원시 데이터와 수치 데이터 구분



- 크기 N은 타입에 포함
- 실행 시간에 비트 수 변경 불가

함수	역할
test(index)	비트 확인, 범위 검사
set(index)	비트 설정
reset(index)	비트 해제
flip(index)	비트 반전
count()	1인 비트 수
any()	하나 이상 설정
none()	모두 0
all()	모두 1

```
#include <bitset>
```

```
std::bitset<8> permissions{0b0010'0101};  
bool firstBit = permissions.test(0);
```

```
permissions.set(1);  
permissions.reset(2);  
permissions.flip(5);  
permissions[0] = false;
```





- 플래그 값은 서로 겹치지 않는 비트로 정의

```
enum class Permission : unsigned int {  
    None    = 0,  
    Read    = 1u << 0,  
    Write   = 1u << 1,  
    Execute = 1u << 2  
};
```

```
constexpr Permission operator|(Permission left, Permission right)  
{  
    return static_cast<Permission>(  
        static_cast<unsigned int>(left) |  
        static_cast<unsigned int>(right));  
}  
constexpr Permission operator&(Permission left, Permission right)  
{  
    return static_cast<Permission>(  
        static_cast<unsigned int>(left) &  
        static_cast<unsigned int>(right));  
}  
constexpr bool HasPermission(Permission value, Permission flag)  
{  
    return (value & flag) == flag;  
}
```

```
Permission permissions =  
    Permission::Read | Permission::Write;
```




- 함수 대부분은 부호 없는 정수 대상
- 부호 있는 정수의 비트 해석에 의존하지 않는 편이 안전

함수	역할
<code>std::popcount()</code>	1인 비트 수
<code>std::has_single_bit()</code>	2의 거듭제곱 여부
<code>std::bit_width()</code>	표현에 필요한 비트 수
<code>std::bit_floor()</code>	값 이하의 가장 가까운 2의 거듭제곱
<code>std::bit_ceil()</code>	값 이상의 가장 가까운 2의 거듭제곱
<code>std::countl_zero()</code>	앞쪽 연속 0 비트 수
<code>std::countr_zero()</code>	뒤쪽 연속 0 비트 수
<code>std::rotl()</code>	왼쪽 순환 이동
<code>std::rotr()</code>	오른쪽 순환 이동

```
include <bit>
#include <cstdint>
std::uint32_t value = 0b0010'1000;
```



- 원본 객체의 비트 표현을 같은 크기의 대상 타입으로 복사
- 대상 타입과 원본 타입의 크기가 같아야 함
- 두 타입은 trivially copyable 조건 필요
- 포인터 별칭을 만들지 않음

```
#include <bit>
#include <cstdint>

float value = 1.0f;
std::uint32_t bits = std::bit_cast<std::uint32_t>(value);
static_assert(sizeof(float) == sizeof(std::uint32_t));
float value = 1.0f;
std::uint32_t numeric = static_cast<std::uint32_t>(value);
std::uint32_t representation = std::bit_cast<std::uint32_t>(value);
```

- static_cast는 수치 변환
- bit_cast는 객체 표현 복사



- 여러 바이트로 구성된 정수의 메모리 배치 순서
- 파일 형식, 네트워크 프로토콜은 바이트 순서를 별도로 정함

값: **0x12345678**

리틀 엔디언: **78 56 34 12**

빅 엔디언: **12 34 56 78**

```
#include <bit>
if constexpr (std::endian::native == std::endian::little)
{
    std::cout << "little endian\n";
}
else if constexpr (std::endian::native == std::endian::big)
{
    std::cout << "big endian\n";
}
else
{
    std::cout << "mixed endian\n";
}
```

- 메모리 값을 그대로 바이트 배열로 복사한다고 직렬화가 끝나지 않음
- 저장 형식의 엔디언에 맞춰 변환 필요
- `std::byteswap()`은 C++23 기능



14.8.7 객체 표현 시 주의점

- 객체의 메모리 바이트 전체를 객체 표현이라고 함
- 구조체 멤버 사이에 패딩 바이트 가능
- 값 표현에 사용되지 않는 패딩 비트 가능

```
struct Data  
{  
    char code;  
    int value;  
};
```

**sizeof(Data)가 sizeof(char) + sizeof(int)와
같다고 보장할 수 없음**

구조체 메모리 직접 저장의 문제

- 컴파일러와 ABI에 따른 패딩 차이
- 엔디언 차이
- 정수와 부동소수점 표현 차이
- 포인터 값과 프로세스 내부 주소
- 클래스 불변식과 객체 수명
- 버전 변경으로 인한 멤버 배치 변화
- 직렬화 형식은 필드 크기, 부호, 바이트 순서, 문자열 인코딩을 명시



- 허용되는 상태를 타입으로 제한
- 호출부에서 값의 의미를 드러냄
- 원시 기본형에 여러 의미를 겹쳐 넣지 않음
- 잘못된 인수 순서와 유효하지 않은 조합을 줄임

```
void OpenWindow(bool border, bool resizable);  
OpenWindow(true, false);
```

true, false의 의미가 호출부에서 불분명

```
enum class BorderMode {  
    Hidden,  
    Visible  
};  
enum class ResizeMode {  
    Fixed,  
    Resizable  
};
```

```
void OpenWindow(BorderMode border, ResizeMode resize);  
OpenWindow(BorderMode::Visible, ResizeMode::Fixed);
```

의미가 타입과 호출부에 드러남



14.9.2 enum class와 의미 타입

```
struct PlayerId
```

```
{  
    int value;  
};
```

```
struct ItemId
```

```
{  
    int value;  
};
```

```
void EquipItem(PlayerId playerId, ItemId itemId);
```

```
PlayerId playerId{100};
```

```
ItemId itemId{200};
```

```
EquipItem(playerId, itemId);
```

```
// EquipItem(itemId, playerId); // 오류
```

- 내부 표현이 같아도 서로 다른 타입
- 인수 순서 오류를 컴파일 시점에 차단 가능

```
class Health
```

```
{
```

```
public:
```

```
    explicit Health(int value) : value(value < 0 ? 0 : value)
```

```
    {
```

```
    }
```

```
    int Get() const
```

```
    {
```

```
        return value;
```

```
    }
```

```
private:
```

```
    int value;
```

- 값 검사와 불변식을 타입 안에 모음



14.9.3 optional, variant와 expected 선택

- "찾지 못함" 만 중요하다면 optional
- 설정 값 종류가 고정되어 있으면 variant
- 실패 이유를 반환값으로 다뤄야 하면 expected
- C++20 기준에서는 std::expected가 표준 기능이 아님

표현하려는 상태	타입
값이 있거나 없음	<code>std::optional<T></code>
정해진 여러 타입 중 하나	<code>std::variant<Ts...></code>
성공값 또는 오류 정보	<code>std::expected<T, E></code>

```
std::optional<Player> FindPlayer(int id);  
using SettingValue = std::variant<int, double, std::string>;
```

```
// C++23
```

```
std::expected<FileData, FileError>  
LoadFile(std::string_view path);
```



14.9.4 유효하지 않은 상태 줄이기

```
// C++23
```

```
std::expected<FileData, FileError>  
LoadFile(std::string_view path);
```

- 성공이면서 시간 초과 가능
- 성공했는데 오류 메시지만 있을 수 있음
- 실패했는데 데이터가 남을 수 있음

```
struct Success
```

```
{  
    std::string data;  
};
```

```
struct Timeout
```

```
{  
    int elapsedMilliseconds;  
};
```

```
struct NetworkError
```

```
{  
    int errorCode;  
};
```

```
using RequestResult = std::variant<Success, Timeout, NetworkError>;
```

- 한 시점에는 한 상태만 활성
- 상태별 데이터도 해당 타입 안에만 존재
- 모순 상태를 타입 구조로 줄임



- `std::pair`는 두 값, `std::tuple`은 여러 값을 하나로 묶음
- 구조적 바인딩의 `auto`, `auto&`, `const auto&`는 복사와 참조 여부를 결정
- 여러 반환값의 의미가 오래 유지되면 이름 있는 구조체가 더 명확
- `std::optional<T>`는 값이 있거나 없는 상태를 표현
- `operator*`, `operator->`는 값 존재를 전제로 함
- `value()`는 값이 없으면 예외
- `value_or()`는 기본값을 값으로 반환
- `std::variant<Ts...>`는 정해진 대안 타입 중 하나를 저장
- `std::visit()`은 활성 타입에 맞는 연산 적용
- `std::any`는 실행 시간에 저장 타입을 결정
- 가능한 타입 집합을 알고 있으면 `variant`가 더 안전





- `std::reference_wrapper<T>`는 참조를 복사 가능한 객체로 감쌈
- `reference_wrapper`는 대상을 소유하지 않음
- `std::span<T>`은 연속 메모리의 포인터와 길이를 묶는 비 소유 범위
- 읽기 전용 `span` 매개변수는 `std::span<const T>`
- 원본 소멸과 재할당은 `span`을 무효화
- `std::string_view`는 문자 구간을 복사하지 않고 참조
- `string_view`는 널 종료를 보장하지 않음
- 임시 문자열과 지역 문자열을 가리키는 뷰는 댕글링 위험
- `std::byte`는 원시 메모리의 한 바이트 표현
- `std::bitset<N>`은 고정 크기 비트 집합
- 객체 메모리 직접 직렬화는 패딩과 엔디언 문제를 일으킬 수 있음
- 타입 안전한 표현은 유효하지 않은 상태 조합을 줄임

